

Evaluación Sumativa 1 — Incremento funcional del backend

Unidad 1 · Aplicaciones web: lado servidor
Modalidad: Trabajo grupal con evidencia individual
Calificación: 10 puntos
Categoría EVA 1: 35 % de la unidad / 3,5 puntos

Propósito

Integrar los aprendizajes desarrollados durante la Unidad 1 mediante la construcción de un **incremento funcional del backend** del proyecto asignado, aplicando desarrollo basado en especificaciones (**SDD**), modelado de procesos y arquitectura, diseño de APIs REST, seguridad, pruebas, documentación técnica, control de versiones y despliegue.

El producto deberá evidenciar correspondencia entre el **problema analizado, los requisitos especificados, la arquitectura propuesta, el contrato de API, la implementación realizada y las pruebas ejecutadas**.

No se busca únicamente presentar código funcional. El equipo deberá demostrar que las decisiones adoptadas pueden justificarse y que el producto desarrollado puede ser comprendido, ejecutado, probado y mantenido a partir de la documentación y evidencias disponibles en el repositorio.

---

1. Producto a entregar

Cada equipo entregará un **repositorio Git accesible** que contenga un incremento funcional del backend del proyecto trabajado durante la unidad.

El repositorio deberá integrar, como mínimo:

A. Especificación y trazabilidad

Evidencias derivadas del trabajo desarrollado durante las semanas 1 y 2:

- problema y contexto del proyecto;
- actores y necesidades;
- alcance definido para el incremento;
- restricciones y riesgos identificados;
- requisitos funcionales;
- requisitos no funcionales;
- historias de usuario o casos de uso;
- criterios de aceptación verificables;
- preguntas de clarificación relevantes;
- relación entre requisitos, implementación y pruebas;
- estructura de trabajo basada en **GitHub Spec Kit**.

Deberá evidenciarse el uso coherente de:

- `constitution`;
- `specify`;
- `clarify`;
- `plan`;
- `tasks`.

Los cambios realizados durante el desarrollo deberán conservar coherencia con las especificaciones. Si una decisión modifica significativamente el alcance o la arquitectura, deberá quedar documentada.

---

2. Modelado del proceso y arquitectura

El equipo deberá presentar los modelos necesarios para comprender el comportamiento y la estructura del backend.

BPMN

Se utilizará **BPMN 2.0.2** para representar al menos uno de los procesos principales del proyecto.

Antes del diagrama deberá existir un breve análisis textual que identifique:

- participantes;
- eventos;
- actividades;
- decisiones;
- flujos;
- datos;
- excepciones relevantes.

Las tareas deberán nombrarse utilizando **verbo en infinitivo + complemento**, por ejemplo:

- Validar solicitud.
- Registrar activo.
- Consultar disponibilidad.
- Aprobar requerimiento.

El modelado podrá incorporar las vistas ejecutiva, operativa o técnica según resulte pertinente al problema.

Arquitectura C4

La arquitectura deberá representarse mediante:

- **C1 — Contexto:** personas, sistema principal y sistemas externos;
- **C2 — Contenedores:** principales aplicaciones, servicios, tecnologías y responsabilidades;
- **C3 — Componentes:** organización interna del backend;
- **C4 — Código:** representación UML de las clases, interfaces y relaciones relevantes del incremento implementado.

Para C1, C2 y C3 se recomienda **Structurizr DSL**.

Para C4 se utilizará preferentemente **PlantUML**.

Los diagramas deben representar la arquitectura del software y sus responsabilidades; no deberán reducirse únicamente a representar tablas de base de datos.

Las decisiones arquitectónicas relevantes deberán documentarse mediante **ADR — Architecture Decision Records**.

---

3. Implementación del backend

El backend deberá implementar un conjunto coherente de funcionalidades derivadas de los requisitos seleccionados para el incremento.

Stack de referencia

- Python.

- Django.
- Django REST Framework.
- PostgreSQL.
- OpenAPI / Swagger.
- pytest / pytest-django.
- Docker.
- Git / GitFlow.
- GitHub Actions.
- Quarto.

La incorporación de tecnologías adicionales deberá justificarse técnicamente.

Organización

Se espera una estructura modular que permita diferenciar, según la solución adoptada:

- API;
- aplicación;
- dominio o lógica de negocio;
- persistencia / infraestructura;
- configuración;
- pruebas.

Se valorará:

- separación de responsabilidades;
- modularidad;
- bajo acoplamiento;
- cohesión;
- aplicación pertinente de principios SOLID.

---

4. Contrato de API

La API deberá:

- utilizar rutas orientadas a recursos;
- aplicar adecuadamente los métodos HTTP;
- utilizar versionado, por ejemplo `/api/v1/`;
- utilizar códigos de estado HTTP coherentes;
- validar parámetros, cuerpos y consultas;
- documentarse mediante **OpenAPI / Swagger**.

Cuando la respuesta HTTP admita cuerpo JSON se utilizará como referencia el contrato universal:

```
{
  "success": true,
  "message": "Operación realizada correctamente.",
  "data": {},
  "errors": null,
  "meta": {}
}
```

El contrato deberá aplicarse consistentemente tanto para respuestas exitosas como para errores.

Deberán respetarse las excepciones propias del protocolo HTTP, como respuestas `204 No Content` o `304 Not Modified`, que no requieren cuerpo.

La utilización de operaciones adicionales o extensiones al contrato deberá justificarse y documentarse.

---

5. Seguridad

La seguridad deberá incorporarse como parte del desarrollo y no como una funcionalidad independiente añadida al final.

El incremento deberá implementar, cuando corresponda:

- autenticación;
- autorización;
- permisos por recurso y/o objeto;
- validación de entradas;
- manejo coherente de errores;
- protección de información sensible;
- separación de secretos y configuración;
- buenas prácticas basadas en recomendaciones OWASP.

El repositorio **no deberá contener contraseñas, tokens, claves privadas ni secretos reales**.

Las decisiones de seguridad relevantes deberán poder relacionarse con los riesgos analizados durante la unidad.

---

6. Pruebas y verificación

Las funcionalidades principales del incremento deberán contar con pruebas automatizadas.

Como mínimo se deberá evidenciar:

- pruebas unitarias;
- pruebas de integración;
- pruebas de endpoints/API;
- validación de contratos y códigos HTTP;
- pruebas de reglas de negocio relevantes;
- comprobaciones básicas de seguridad.

Las pruebas deberán demostrar el cumplimiento de los criterios de aceptación definidos.

No se considerará terminada una funcionalidad cuando exista código implementado pero sus condiciones principales no puedan verificarse mediante pruebas o evidencia reproducible.

El equipo deberá poder ejecutar las pruebas mediante instrucciones claramente documentadas.

---

7. Control de versiones y colaboración

El repositorio deberá evidenciar el proceso real de desarrollo del equipo.

Se utilizará **GitFlow** o una estrategia equivalente previamente justificada.

Se verificará:

- ramas coherentes con la estrategia adoptada;
- ramas de funcionalidades;
- integración mediante Pull Requests cuando corresponda;
- mensajes de commit comprensibles;
- historial de contribuciones;
- revisión e integración progresiva;
- ausencia de cambios masivos sin trazabilidad.

El historial de Git constituye parte de la evidencia de participación individual.

---

8. Documentación técnica

Se utilizará **Quarto como portal central de documentación** del proyecto.

La documentación deberá integrar, según corresponda:

- descripción del problema;
- alcance;
- actores;
- requisitos;
- procesos BPMN;
- arquitectura C4;
- UML;
- ADR;
- contrato OpenAPI;
- reglas de negocio;
- seguridad;
- pruebas;
- instrucciones de instalación;
- configuración;
- ejecución;
- despliegue;
- bitácora o evolución del proyecto.

El `README.md` deberá permitir que una persona externa al equipo comprenda cómo preparar y ejecutar el proyecto.

La documentación deberá corresponder al estado real de la implementación.

---

9. Contenerización y despliegue

El proyecto deberá disponer de instrucciones reproducibles para ejecutar el backend.

Se utilizará **Docker** cuando corresponda al entorno definido para la asignatura.

La entrega deberá permitir identificar:

- dependencias;
- variables de entorno;

- configuración necesaria;
- base de datos;
- procedimiento de inicialización;
- ejecución del backend;
- ejecución de pruebas;
- acceso a la documentación de API.

Se valorará positivamente la automatización mediante **GitHub Actions** de procesos como:

- ejecución de pruebas;
- análisis estático;
- validaciones de calidad;
- construcción del proyecto.

No se exige infraestructura de producción compleja.

---

10. Observabilidad mínima

El incremento deberá incorporar mecanismos básicos que faciliten identificar su comportamiento durante la ejecución.

Como mínimo:

- registros de aplicación;
- manejo estructurado de errores;
- identificación de operaciones relevantes;
- mecanismo básico para comprobar que el servicio se encuentra disponible.

Cuando el proyecto lo justifique podrá incorporar adicionalmente métricas, health checks, tiempos de respuesta u otros mecanismos de observabilidad.

Evidencias de entrega

El equipo deberá entregar en EVA:

1. Enlace al repositorio

Repositorio accesible que contenga el código, especificaciones, pruebas y documentación.

2. Enlace a la documentación técnica

Portal Quarto o documentación equivalente generada desde el repositorio.

3. Demostración funcional

El equipo realizará una demostración breve en la que deberá evidenciar:

1. problema abordado;
2. especificaciones seleccionadas;
3. arquitectura;
4. API;
5. funcionalidad principal;
6. autenticación/seguridad;

7. pruebas;
8. documentación;
9. procedimiento de ejecución o despliegue.

La demostración deberá realizarse sobre el producto desarrollado y no exclusivamente mediante diapositivas.

4. Evidencia de contribución individual

Cada estudiante deberá registrar:

- funcionalidades o componentes en los que participó;
- Pull Requests, commits u otras evidencias verificables;
- decisión técnica o problema en cuya resolución intervino;
- pruebas o documentación desarrolladas;
- breve reflexión sobre su contribución al incremento.

El estudiante deberá poder explicar técnicamente el trabajo declarado.

La existencia de un producto grupal **no garantiza automáticamente la misma calificación individual** cuando no exista evidencia suficiente de participación.

Correspondencia con las actividades de la Unidad 1

Semana	Evidencia previa	Integración en la evaluación sumativa
1	Problema, actores, SDD, repositorio y entorno	Contexto, alcance, Git, Spec Kit y organización inicial
2	Requisitos, `Specification.md`, historias y trazabilidad	Especificación y criterios de aceptación
3	Arquitectura, ADR, OpenAPI y endpoints base	BPMN, C4, UML, contrato e implementación
4	Modelo de amenazas, autenticación, validaciones y pruebas	Seguridad y verificación
5	Integración, despliegue, dossier técnico y reflexión	Incremento integrado, documentación y demostración

Por tanto, la evaluación sumativa **no requiere reconstruir desde cero el proyecto**. El equipo deberá consolidar, corregir e integrar los productos desarrollados durante toda la unidad.

Condición de funcionalidad mínima

El incremento presentado deberá permitir ejecutar de extremo a extremo al menos uno de los flujos principales especificados para el proyecto.

Por ejemplo:

requisito → solicitud HTTP → endpoint → validación → regla de negocio → persistencia → respuesta → prueba

No será suficiente entregar únicamente:

- modelos;
- documentación;
- interfaces simuladas;
- endpoints sin lógica;
- respuestas `mock`;
- código que no pueda ejecutarse o verificarse.

Rúbrica de evaluación

Criterio	2 — Logro esperado	1 — En proceso	0 — No evidenciado
1. Especificación, trazabilidad y arquitectura	Existe correspondencia verificable entre problema, actores, requisitos, criterios de aceptación, BPMN, arquitectura C4/UML, ADR e implementación. Las decisiones son coherentes y justificadas.	Los principales artefactos existen, pero presentan inconsistencias, trazabilidad parcial o decisiones insuficientemente justificadas.	La implementación no puede relacionarse de forma clara con los requisitos y arquitectura, o faltan evidencias esenciales.
2. Funcionamiento y diseño de la API	El incremento funciona de extremo a extremo; los endpoints implementan reglas de negocio pertinentes, utilizan correctamente HTTP, REST, versionado, contrato universal y documentación OpenAPI.	La funcionalidad principal opera parcialmente o existen inconsistencias en endpoints, métodos, códigos HTTP, contrato o documentación.	La API no funciona adecuadamente, depende principalmente de respuestas simuladas o carece de coherencia con los requisitos.
3. Seguridad y calidad de implementación	Se aplican autenticación, autorización, validación, manejo de errores, protección de secretos y buenas prácticas pertinentes; la estructura es modular y mantenible.	Existen medidas básicas de seguridad y organización, pero son incompletas, inconsistentes o presentan problemas de diseño.	No existen controles mínimos de seguridad o la implementación presenta deficiencias críticas que comprometen su funcionamiento o mantenibilidad.
4. Pruebas y verificación	Las funcionalidades y reglas críticas disponen de pruebas automatizadas pertinentes; se verifican criterios de aceptación, API, integración y casos relevantes de error.	Existen pruebas, pero su cobertura funcional es limitada o no permiten verificar claramente todos los comportamientos principales.	Las pruebas son inexistentes, meramente demostrativas o no permiten verificar el comportamiento del incremento.

5. Documentación, despliegue y contribución	Las pruebas son reproducibles. README, Quarto, OpenAPI e instrucciones permiten comprender, ejecutar, probar y desplegar el proyecto. Git evidencia trabajo colaborativo y cada integrante demuestra una contribución verificable.	La documentación o el procedimiento de ejecución son parcialmente reproducibles, o existen debilidades en la evidencia de colaboración/contribución.	La documentación es insuficiente, el proyecto no puede reproducirse con las instrucciones entregadas o no existe evidencia clara de contribución.

Puntaje máximo: 10 puntos

Cada criterio tendrá un valor máximo de **2 puntos**.

Conversión:
10/10 en la evaluación corresponde al máximo de **3,5 puntos de la categoría EVA 1**.

Uso de inteligencia artificial

Se permite el uso responsable de herramientas de inteligencia artificial como apoyo al proceso de desarrollo.

El equipo deberá:

- declarar las herramientas utilizadas;
- identificar de manera general para qué fueron empleadas;
- verificar técnicamente las respuestas generadas;
- mantener responsabilidad sobre código, modelos, pruebas y documentación;
- citar las fuentes externas utilizadas cuando corresponda.

La generación automática de código o documentación **no sustituye la capacidad del estudiante para explicar, justificar y modificar el producto presentado**.

Durante la demostración, cualquier integrante podrá ser consultado sobre los componentes en los que haya declarado participación.

Criterio de finalización

Una funcionalidad será considerada terminada cuando:

- corresponda a un requisito especificado;
- cumpla sus criterios de aceptación;
- respete las reglas de negocio;
- sea coherente con la arquitectura;
- utilice el contrato API definido;
- incorpore las validaciones y controles de seguridad pertinentes;
- disponga de pruebas satisfactorias;
- pueda ejecutarse e integrarse;

- mantenga actualizada su documentación;
- exista evidencia de su incorporación al flujo de trabajo del equipo.

Principio rector: el propósito de la evaluación no es medir cuántas líneas de código fueron desarrolladas, sino comprobar que el equipo puede construir un backend cuya estructura, comportamiento, seguridad, calidad y evolución puedan justificarse y verificarse a partir de especificaciones explícitas.